

SOFTVETClinic – Software gestión clínicas veterinarias

Se te ha encomendado desarrollar una pequeña aplicación para la gestión de clínicas veterinarias, sin embargo, sólo vas a desarrollar una parte de su funcionalidad porque se tratará de la versión demo o “shareware”.

La aplicación funcionará en modo consola de comandos y se gestionará mediante menús, con un aspecto limpio y presentable. Se adjunta el diagrama de clases y las especificaciones a cumplir en cuanto a relaciones y cardinalidades; sin embargo, el diagrama no está completo ya que faltan muchos métodos que tendrás que crear e implementar en el lugar que corresponda para dar cumplimiento a las funciones que se piden. Las variables que no indican tipo de datos son String.

Desde el programa principal tendrás que poder almacenar todas las personas, animales, actuaciones y facturas existentes. Además, queremos que se puedan realizar estas funciones:

- **Alta y baja de clientes:** no permitas duplicados por nombre. Para eliminar, debemos poder hacerlo buscando por nombre. No permitas la baja de un cliente si tiene animales asociados, deben eliminarse primero los animales.
- **Alta y baja de animales:** no permitas duplicados por nombre. La baja será buscando por nombre.
- **Crear actuación y actualizar actuación.** Los estados posibles para una actuación sólo pueden ser: “Iniciada”, “En proceso”, “Finalizada”.
- **Generar factura:** se pedirá identificar la actuación (que deberá figurar en estado “Finalizada”, de lo contrario no se podrá).
- **Obtener facturación mensual:** pedirá un mes en número (1 al 12) y obtendrá la suma del importe de las facturas únicamente de ese mes.
- Debemos poder **mostrar** en pantalla la información (limpia y bien presentada) de cualquier elemento del sistema (Personas, Animales, Actuaciones y Facturas), y queremos poder hacerlo en listado completo o bien buscando un elemento individual por el campo o atributo identificado que consideres más adecuado. Los clientes mostrarán también los datos de sus animales.
- `pagarFactura()`: debe pedir la factura y actualizar su estado.
- `finalizarActuacion()`: debe actualizar el estado de una actuación a “Finalizada”.

Aclaraciones para atributos:

- Las fechas tendrán que ir en formato dd/mm/aaaa.

Debes implementar las funciones de forma robusta (tolerantes a entradas erróneas) y que requieran la información necesaria para poder funcionar. La aplicación debe informar del resultado de las operaciones (especialmente si hay algún fallo, datos inexistentes, etc.).

Es obligatorio seguir las buenas prácticas y guías de diseño del paradigma orientado a objetos, así como de la modularidad.

Diagrama de clases del sistema

